

■ CORE CONCEPTS

Component

JS function returning JSX. UI = f(props,state). Reusable, independent.

Virtual DOM

Lightweight JS object. React diffs old vs new → patches only changed real DOM nodes.

Reconciliation

Compare old/new V-DOM → compute minimal real DOM operations needed.

Fiber

React 16+ engine. Units of work — pauseable, prioritised, resumable.

Declarative

Describe WHAT the UI looks like for a state. React handles HOW.

JSX

Babel compiles JSX → React.createElement() → V-DOM object. Browser never sees JSX.

```
className htmlFor camelCase <>fragment
{expressions only – no if/else statements inside {}}
```

■ PROPS vs STATE

Owned by	Props=parent State=component itself
Mutable?	Props=read-only State=yes via setter
Re-renders	Both trigger re-render when changed
One-way	Data flows parent→child via props
Pass up	Callback fn as prop — child calls it

```
const [val,setVal] = useState(0);
setVal(prev=>prev+1); //functional update
setVal({...val,key:'new'}); //objects: spread
```

■ **Never mutate state — always create new value**

■ **Init strings with " not undefined (forms!)**

■ **Functional update when new state needs old**

■ ALL HOOKS — ONE LINE EACH

useState	Component memory. [val,setter]. Triggers re-render on update.	useRef	Mutable box. DOM access or silent storage. NO re-render on change.
useEffect	Side effects after render. 3 forms: no-dep / [] / [dep].	useCallback	Memoize FUNCTION. Stable ref. Pair with React.memo on child.
useContext	Read from Context. Eliminates prop drilling. Re-renders on change.	useMemo	Memoize VALUE. Expensive calc or stable obj/arr ref for child.
useReducer	dispatch(action)→reducer(state,action)→newState. Stable dispatch.	useId	Stable SSR-safe unique ID. For htmlFor/id pairs. Not list keys.
useTransition	React 18. [isPending,startTransition]. Non-urgent updates.	useDeferredValue	React 18. Defer received value. Use when you don't own setter.

■ useEffect

```
useEffect(()=>{...}); // every render
useEffect(()=>{...},[]); // mount only
useEffect(()=>{...},[dep]); // dep changed
useEffect(()=>{return cb;},[]); //cleanup
```

Key misconceptions:

- Runs AFTER paint — not during render
- [] = no deps, not 'once' (StrictMode runs twice)
- Cannot be async — wrap async function inside
- Object in deps = infinite loop (new ref each render)
- Cleanup runs before NEXT effect too, not just unmount
- Data transforms don't belong in useEffect

■ RULES + CONTEXT

Rule 1	Hooks at TOP LEVEL only — no loops or conditions
Rule 2	Only in React components or custom hooks (use*)
Stale closure	Missing dep → captures stale value bug

```
const Ctx = createContext(null);
val,[v]]>
// Consumer: useContext(Ctx)
// Custom hook pattern:
function useCtx(){
const c=useContext(Ctx);
if(!c) throw Error('Need Provider');
return c;}

```

■ PERFORMANCE TRIO

```
// All 3 work together – missing one defeats rest
const C=React.memo(({val})=>); //memo
const fn=useCallback(()=>{,[deps]}); //fn stable
const v=useMemo(()=>compute(),[d]); //val stable
```

■ Measure with DevTools Profiler before fixing!

- Virtualise lists >100 items → react-window
- Split contexts: one per concern (auth/theme/cart)
- Colocate state: keep as low in tree as needed
- Debounce search, throttle scroll/resize

■ GOTCHAS — Know Cold

{0&&}	Renders '0'! Use length>0&&
key=index	Never if list reorders/deletes
HOC in render	Creates new type → remounts every render
async useEffect	Define async inside, call it
dispatch stable	useReducer dispatch never changes ref
useState(undef)	Switches uncontrolled→controlled. Use ""
Object in deps	New ref every render = infinite loop
Cleanup timing	Runs before NEXT effect AND unmount

■ REACT ROUTER v6		■■ ERROR BOUNDARY + PORTAL	
Setup	BrowserRouter→Routes→Route path=" element={}	getDerivedStateFromError	Update state → show fallback (class only)
Link	Link to=" declarative NavLink adds isActive	componentDidCatch	Log to Sentry/error tracking service
Navigate	useNavigate() navigate('/p') (-1) replace:true	Cannot catch	Event handlers, async, SSR, self-errors
Params	path="/users/:id" → const {id}=useParams()	3 layers	Top + Route + Widget level boundaries
Query	[p,set]=useSearchParams() p.get('k') set({k:'v'})	Library	react-error-boundary — production standard
Nested	Outlet in parent — child routes render inside	// Portal – escapes overflow:hidden createPortal(children, document.body) // Events still bubble through React tree // Use: modals, tooltips, toasts, dropdowns	
Protected	if(!user) return Navigate to="/login" replace		
Lazy	lazy(()=>import('./P'))+Suspense fallback={}		
useLocation	pathname,search,hash,state — page tracking		

■ HOC · RENDER PROPS · HOOKS		■ LISTS · KEYS · CONDITIONAL	
HOC	withAuth(C) — enhanced component. Spread props. displayName. Create outside render.	map()	Only way to render lists. key on root element.
Render Props	Mouse render={pos=><Cat/>} or children as function	key rules	Unique among siblings. Stable ID. Never index.
Custom Hook	use* prefix. Can use hooks. Own state per call.	ternary	{cond?<A/>:} — show A or B
Prefer	Custom Hooks — flat, no wrappers, no prop collision	&&	{cond&&<A/>} — show or nothing. Use cond>0
■ CODE SPLITTING		early return	if(loading) return Spin — multi-condition
		??	{val??'default'} — only null/undefined not 0

const P=lazy(()=>import('./Page')); > ... // Preload on hover: import('./Page')		■■ {items.length&&<L/>} renders '0'! Fix: length>0&&	
		■ FORMS	
		Controlled	value+onChange. React owns. Re-renders on type.
		Uncontrolled	defaultValue+ref. DOM owns. Read on submit.
		Checkbox	Use checked+e.target.checked (not value)
		File input	Always uncontrolled — browser security
		RHF	Uncontrolled perf + controlled-like API

■ REACT 18 FEATURES		■ useREDUCER + STATE MANAGEMENT	
createRoot	REQUIRED. createRoot(el).render(<App/>)	Syntax	const [state,dispatch]=useReducer(reducer,init)
Auto Batching	Multiple setState = 1 re-render everywhere incl async	When	3+ related fields updating together
useTransition	[isPending,startTrans] — non-urgent updates	vs useState	Identical perf. useState IS useReducer internally.
useDeferredValue	Defer received value — prioritise urgent renders	dispatch	Always stable ref — no useCallback needed
T vs D	useTransition=own setter useDeferredValue=prop	Mini Redux	useReducer + useContext = no extra library
Suspense data	Works with React Query — no manual loading state	■■ STATE MANAGEMENT	
Streaming SSR	Progressive HTML. loading.tsx in Next.js	Props	1–2 levels deep — simplest and explicit
flushSync	Force immediate re-render — escape batching (rare)	useContext	Global UI: theme, auth, language
		High-freq	Mouse/scroll → local state, NOT context
		Zustand	Small-medium: minimal, no Provider needed
		Redux	Large teams: DevTools + strict patterns

Hook→Class Lifecycle		JSON Quick Ref		Must-Know One-Liners
componentDidMount	useEffect(fn,[])	stringify	Object→String. localStorage, fetch body	- React = library not framework. View layer only. - UI = f(state,props). Same input = same output. - Virtual DOM: diff → patch changed nodes only. - Hooks share LOGIC not STATE per call instance. - Error Boundaries: class only, no hook equiv yet. - createRoot() required for React 18 concurrent.
componentDidUpdate	useEffect(fn,[dep])	parse	String→Object. localStorage read	
		res.json()	Auto-parses fetch response	
componentWillUnmount	useEffect(()=>{return cb},[])	Deep clone	JSON.parse(JSON.stringify(obj))	
getDerivedStateFromError	class only — ErrorBoundary	structuredClone	Better clone — handles Date/Map/Set	
shouldComponentUpdate	React.memo			

■ CONCURRENT RENDERING (React 18)		■ AUTO BATCHING + useId + SSR	
React 17	Sync: A starts→finishes→B starts. Cannot interrupt.	<pre>// React 17 – only event handlers batched setTimeout(()=>{ setA(1); // re-render 1 ■ setB(2); // re-render 2 ■ },0) // React 18 – everywhere automatically setTimeout(()=>{ setA(1); // batched ■ setB(2); // ONE re-render total ■ },0)</pre>	
React 18	Concurrent: A starts→urgent B→A paused→B done→A resumed		
createRoot	Enables concurrent mode. Required for all React 18.		
startTransition	Non-urgent. React can interrupt for urgent work.		
isPending	true while transition resolves — show inline spinner		
useDeferredValue	Let value lag — keep input responsive		
<pre>// useTransition – you own the update const [pending,startT]=useTransition(); startT(()=>setResults(filter(val))); // useDeferredValue – value from props/parent const dv=useDeferredValue(query); const res=useMemo(()=>filter(dv),[dv]); const stale=query!==dv; // dims while catching up</pre>		useId	Stable SSR-safe ID for htmlFor/id pairs. Not list keys.
		Streaming SSR	Server streams HTML progressively — no full wait
		loading.tsx	Next.js: automatic Suspense boundary per route
		flushSync	Force sync re-render to escape batching (rare)

■ INTERVIEW Q&A; BANK — 15 Key Questions with Model Answers

What is React?	Functional vs Class?	What is Virtual DOM?
JS library for UIs using components. Virtual DOM diffing updates only changed nodes. Declarative: describe WHAT, not HOW.	Functional: plain function + Hooks. Class: extends Component + lifecycle + this. Functional preferred today.	Lightweight JS object mirroring real DOM. React diffs old vs new, computes minimal changes, patches real DOM.
Why not mutate state?	What is stale closure?	useCallback vs useMemo?
React uses reference equality. Same ref = no change detected = no re-render. Always create new value/object.	Effect captures variable at creation, never updates. Fix: add to deps array or use functional state updates.	useCallback=memoize FUNCTION. useMemo=memoize VALUE. Both need React.memo on child to prevent re-renders.
React.memo not working?	Prop drilling fix?	useReducer vs useState?
Unstable prop ref. Object/array/fn created inline = new ref each render = memo fails. Fix: useMemo+useCallback.	Context API: createContext+Provider+useContext. Split into multiple contexts. One context per concern.	Same performance. useState IS useReducer. Choose by clarity: 1-2 simple=useState, 3+ related=useReducer.
What are Error Boundaries?	What is code splitting?	React 18 biggest change?
Class component catching render errors. getDerivedStateFromError=fallback, componentDidCatch=log. Not async/events.	React.lazy+import() splits bundle into chunks. Load on demand. Route-level split = biggest performance win.	Concurrent rendering: pause/interrupt/resume. createRoot() required. Automatic batching everywhere.
Controlled vs Uncontrolled?	HOC vs Custom Hooks?	What is Fiber?
Controlled: value+onChange, React owns. Uncontrolled: defaultValue+ref, DOM owns. File input always uncontrolled.	Both share logic. HOC wraps component, injects props. Custom Hook: no wrapper, no prop collision, composable. Prefer hooks.	React 16+ rendering engine. Splits render into small pauseable units. Enables concurrent features + prioritisation.